

UD6. ABC's DE JAVA.

1. Características generales de Java.
 - A. Características.
 - B. Palabras reservadas.
 - C. Tipos de datos.
 - D. Literales.
2. Estructura de un programa.
3. Instrucciones básicas.
 - A. Instrucciones de definición de datos.
 - a) Identificadores.
 - B. Instrucciones primitivas.
 - a) Asignación.
 - b) Salida.
 - c) Entrada.
4. Operadores.

UD6. ABC's DE JAVA.

5. Procedimiento para escribir un algoritmo en Java.
6. Instrucciones compuestas.
7. Instrucciones de control.
 - A. Instrucción de bifurcación incondicional.
 - B. Instrucciones alternativas.
 - a) Alternativa Simple.
 - b) Alternativa doble.
 - c) Alternativa múltiple.
 - C. Instrucciones repetitivas.
 - I. Mientras.
 - II. Repetir.
 - III. Para.

1.A. Características generales de java

- ☐ Distingue entre mayúsculas y minúsculas.
- ☐ Cada instrucción termina en un ;
- ☐ {...} delimitan un bloque de instrucciones
- ☐ // comentario fin de línea
- ☐ /* */ comentario párrafo
- ☐ /** */ comentario para documentar.

1. Características generales de java

B. Palabras reservadas

abstract	boolean	break	byte	case
catch	char	class	const*	continue
default	do	double	else	extends
final	finally	float	for	goto*
if	implements	import	instanceof	int
interface	long	native	new	null
package	private	protected	public	return
short	static	super	switch	this
synchronized	throw	throws	transient	try
void	volatile	while	true	false

1. Características generales de java

C. Tipos primitivos

Tipo	Descripción	Rango
byte	Datos enteros (1 byte)	Desde -128 al 127
short	Datos enteros (2 bytes)	Desde -32.768 al 32.767
int	Datos enteros (4 bytes)	Desde -2.147.483.648 al 2.147.483.647
long	Datos enteros (8 bytes)	Desde -9.223.372.036.854.775.808 al 9.223.372.036.854.775.
float	Datos en coma flotante (4 bytes) (1 bit signo, 8 exponente, 24 para la mantisa)	Desde 1×10^{-45} al 3×10^{38}
double	Datos en coma flotante (8 bytes) (1 bit signo, 11 exponente, 52 para la mantisa)	Desde 4×10^{-324} al 1×10^{308}
char	Datos enteros y caracteres	Carácter (0-127)
boolean	Valores booleanos.	true/false

PROG.

Nuria Fuentes Pérez



1. Características generales de java

D. Literales

Literales numéricas.

4 (byte, short y int).

4L (long).

-4 (byte, short y int)

2.56 (double).

2.56F (float)

256E-2 (double)

Literales cadena.

Son objetos, no son un tipo primitivo y se encierran entre comillas dobles.

Pueden contener cualquier carácter.

Literales booleanas.

true

false

Literales carácter.

'a'

\n → Línea nueva

\t → Tabulador

\b → Retroceso

\r → Retorno de carro

\f → Salto de hoja

\\ → contrabarra

\' → Comilla simple

\\" → Comillas dobles

PROG.

Nuria Fuentes Pérez



2. Estructura general de un programa

Estructura de un programa en Pseudocódigo

Programa: <NombrePrograma>

Autor: <Nombre Apes>

Entorno:

// ***** declaración de variables

Algoritmo:

//***** Secuencia de instrucciones

Fin Programa

2. Estructura general de un programa

Estructura de una aplicación Java

```
class nombreClase{  
    //Cuerpo de la clase  
}
```

```
//Programa: NombrePrograma  
//Autor: Nombre Apes  
public class nombreClase{  
    public static void main(String[] args){  
        //Entorno:  
        // ***** declaración de variables  
        //Algoritmo:  
        // ***** Secuencia de instrucciones  
    } //Fin Programa  
}
```

PROG.

Nuria Fuentes Pérez



3. Instrucciones básicas

- **Instrucciones de definición de datos.**
- **Instrucciones primitivas.**
 - **De asignación.**
 - **De entrada.**
 - **De salida.**

3. Instrucciones básicas

A. Instrucciones de definición de datos

Son aquellas instrucciones usadas para informar al procesador del espacio que debe reservar de memoria, para almacenar datos durante la ejecución de un programa.

La definición consiste en:

- indicar un identificador para referenciar al dato
- indicar un tipo de datos para informar al procesador de características y espacio a reservar.

Los datos a definir son: variables o constantes.

3. Instrucciones básicas

A. Instrucciones de definición de datos

- Variables de tipos primitivos o básicos.
- Variables referencia **POO**

Desde el punto de vista del papel que juegan en el programa, pueden ser:

- Variables de clase. **POO**
- Variables de instancia. **POO**
- Variables locales.

3. Instrucciones básicas

A. Instrucciones de definición de datos a)Identificadores

➤Identificadores:

- Empezar por letra, carácter subrayado (_) o \$.
- No puede empezar por carácter numérico.
- Después del primer carácter puede venir cualquier combinación de letras y números.

NOTACIÓN:

- 1ª letra del nombre de variable minúsculas.
- Cada palabra después de la primera letra empieza por mayúsculas.
- Resto en minúsculas.

3. Instrucciones básicas

A. Instrucciones de definición de datos (VARIABLES)

<nVariable>[,<nVariable2...] es {
numérico { entero
real }
alfanumérico (max)
booleano }

tipo nvariable [=valor] [, nvariable [= valor]...];

Inicialización:

- Variables locales: ERROR compilación
- Variables clase/instancia: 0 '\0' false null

int edadAlumno, x=23;

Yeti j;

float sueldoCliente=0;

POO

PROG.

Nuria Fuentes Pérez



3. Instrucciones básicas

A. Instrucciones de definición de datos (VARIABLES)

Ejemplos:

```
int numAlumno, x=23; // declaración y asignación NO PERMITIDO
```

```
Yeti j;
```

```
float sueldo;
```

edadAlumno,x es numérico entero

sueldoCliente es numérico real

```
byte edadAlumno;
```

```
int x;
```

```
float sueldoCliente;
```

PROG.

Nuria Fuentes Pérez



3. Instrucciones básicas

A. Instrucciones de definición de datos (CONSTANTES)

CONST <NCONSTANTE> **es** {
 numérico { entero
 real
 alfanumérico
 booleano
} ← <valor>

final tipo nvariable =valor;

CONST PI es numérico real ← 3.1416

final float PI=3.1416F;

PROG.

Nuria Fuentes Pérez



3. Instrucciones básicas

B. Instrucciones primitivas

➤ De asignación

Son aquellas instrucciones encargadas de almacenar un valor en la dirección de memoria referenciada por un identificador.

`<nVariable> ← <expresión>`

La expresión puede ser:

- constante
- literal
- variable
- expresión

`importeBruto ← 1100`

`importeNeto ← importeBruto * 1.16`

Pero deben ser **ambas** del mismo tipo de datos

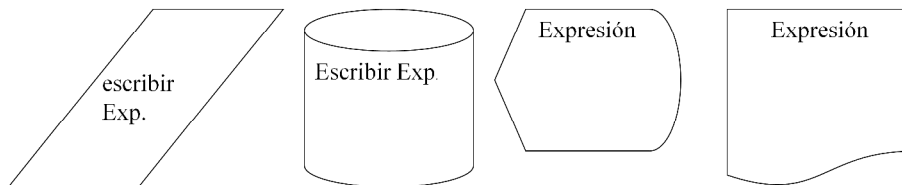
`nvariable = valor;`

3. Instrucciones básicas

B. Instrucciones primitivas

➤ De salida

Son aquellas destinadas a recoger datos de la posición de memoria indicada y depositarlos en dispositivos de salida.



Escribir <Expresión1>[, <Expresión2>...]

3. Instrucciones básicas

B. Instrucciones primitivas

➤ De salida

En java no hay instrucciones básicas para la entrada ni la salida.
La salida se realiza mediante el objeto `System.out`

```
System.out.print(cadena);  
System.out.println(cadena);
```

Envía la cadena a la salida estándar

```
System.out.println("Hola mundo");
```

```
System.out.print("Hola "+"Mundo.");
```

Envía la cadena a la salida estándar y añade un fin de línea

```
int x=24;
```

```
System.out.print("El resultado es "+x);
```

3. Instrucciones básicas

B. Instrucciones primitivas

Qué salida tiene.....

```
//Programa: pruebaSalida
class pruebaSalida{
    public static void main(String[] args) {
        //Entorno:
        //Algoritmo:
        System.out.print("Me llamo \"Nemo\" el grande");
        System.out.print("\nDirección: C\\ Mayor 25");
        System.out.print("\nHa salido la letra \"L\"");
        System.out.print("\n\tEsto ha sido todo");
        System.out.print("\nRetrosceso\b89");
    } //Fin Programa
}
```

PROG.

Nuria Fuentes Pérez

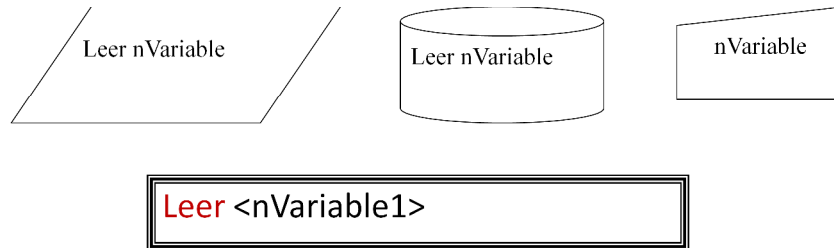


3. Instrucciones básicas

B. Instrucciones primitivas

➤ De entrada

Son aquellas destinadas a recoger el dato de un periférico de entrada y almacenarlo en memoria en la posición indicada por la variable.



3. Instrucciones básicas

B. Instrucciones primitivas

➤ De entrada

En java no hay instrucciones básicas para la entrada ni la salida. La entrada se realiza mediante objetos que manejan la entrada desde los diferentes dispositivos.

Nosotros, de momento, usaremos la clase **Leer**, especialmente construida para recibir información del teclado.

```
Leer.dato()
Leer.datoShort()
Leer.datoInt()
Leer.datoLong()
Leer.datoFloat()
Leer.datoDouble()
Leer.datoChar()
```

```
short edad; //en Entorno
edad=Leer.datoShort(); //en Algoritmo
```

4. Operadores

➤ Aritméticos

Operador aritmético	Significado
+	Suma
-	Resta
*	Multiplicación
/	División
%	Resto

Operador concatenación	Significado
+	Concatenación

```
short x,y; //en Entorno  
int z; //en Entorno  
x=20; //en Algoritmo  
y=3; //en Algoritmo  
z = x+y; //en Algoritmo
```

La mayoría de las operaciones que involucran enteros producen un **int** sin importar el tipo original de los operandos. Si se trabaja con otros tipos tendríamos que asegurarnos de que los operandos tengan el mismo tipo con el que se quiere terminar.

PROG.

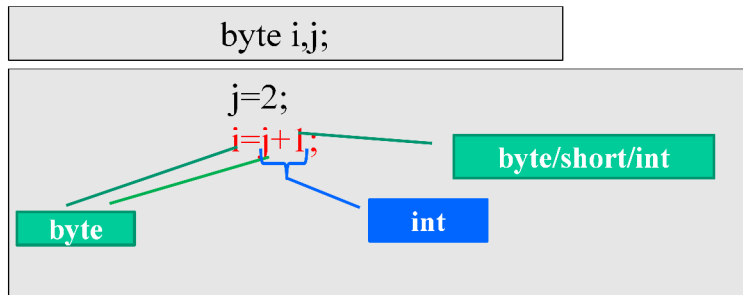
Nuria Fuentes Pérez



4. Operadores

➤Casteo

(tipo) expresión



Error compilación!!!: posible pérdida de precisión

4. Operadores

➤ Relacionales

Operador	Significado
==	igual
!=	Distinto
<	Menor
>	Mayor
<=	Menor o igual
>=	Mayor o igual

```
boolean hipo; //en Entorno
int edad; //en Entorno
edad=31; //en Algoritmo
hipo = (edad < 31); //en Algoritmo
```


4. Operadores

➤ Lógicos

Operador	Significado	resultado
&&	And	true si op1 y op2 son true. Si op1 es false ya no se evalúa op2
&	And	true si op1 y op2 son true. Siempre se evalúa op2
	Or	true si op1 u op2 son true. Si op1 es true ya no se evalúa op2
	Or	true si op1 u op2 son true. Siempre se evalúa op2
^	Xor	True si op1 y op2 tienen distinto valor
!	Not	true si op es false y false si op es true

4. Operadores

➤ Asignaciones especiales

Expresión	Significado
$x += y$	$x = x + y$
$x -= y$	$x = x - y$
$x *= y$	$x = x * y$
$x /= y$	$x = x / y$
$x \% = y$	$x = x \% y$

```
x = x / y + 5  
x /= y + 5
```

Probarlo para $x=20$ y $y=5$

Operadores de sufijo y postfijo	
++	--

```
x++;  
++x;  
x--;  
--x;
```

```
x = 40;  
y = x++;  
z = ++x;
```

4. Operadores PRIORIDAD

Operadores posfijos `expr++` `expr--`

Operadores unarios `++expr` `--expr` `+expr` `-expr` `!`

Creación o cast `new (type)expr`

`*` `/` `%`

`+-`

relacionales `<` `>` `<=` `>=` `instanceof`

igualdad `=` `!=`

`&`

`^`

`|`

`&&`

`||`

condicional `?:` (condición? entonces:sino;)

asignación `=` `+=` `-=` `*=` `/=` `%=`

PROG.

Nuria Fuentes Pérez



5. Procedimiento para escribir un algoritmo en java

1. DATOS.
 - a. Estudiar qué datos hay que obtener y qué datos se necesita para obtenerlos, indicando qué información tienen, qué dominio de valores pueden tener.
 - b. Estudiar cómo serán tratados cada datos en el algoritmos:
 - i. Constantes (Tipificadas o Literales)
 - ii. Variables
2. SOLUCIÓN
 - a. Solucionar el problema con bolígrafo y papel. (Obtener el resultado)
 - b. Escribir, de forma detallada, los pasos a dar para solucionar el problema.
3. Escribir el algoritmo (todo lo anterior) usando la técnica de Pseudocódigo.
4. Ejecutar el algoritmo para comprobar que hace lo que pretendíamos. (Traza)
5. Escribir el pseudocódigo usando sintaxis Java.

PROG.

Nuria Fuentes Pérez



6. Instrucciones compuestas

•Llamada al módulo

Son aquellas instrucciones que no pueden ser directamente ejecutadas por el procesador ya que están constituidas por un bloque de instrucciones agrupadas en módulo o subprograma.

	Llamada al módulo	
--	----------------------	--

En Java:
System.out.print(argumentos o parámetros)
Leer.dato()

7. Instrucciones de control

A. Instrucción de bifurcación incondicional

Son utilizadas para controlar la secuencia de ejecución de un algoritmo.

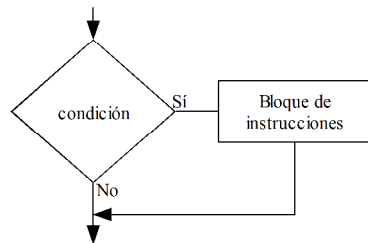
•Instrucción de salto o bifurcación incondicional.

Son aquellas instrucciones que alteran la secuencia normal de ejecución de un algoritmo sin ninguna dar opción a ninguna toma de decisión.

7. Instrucciones de control

B. Instrucciones alternativas

➤ Alternativa simple



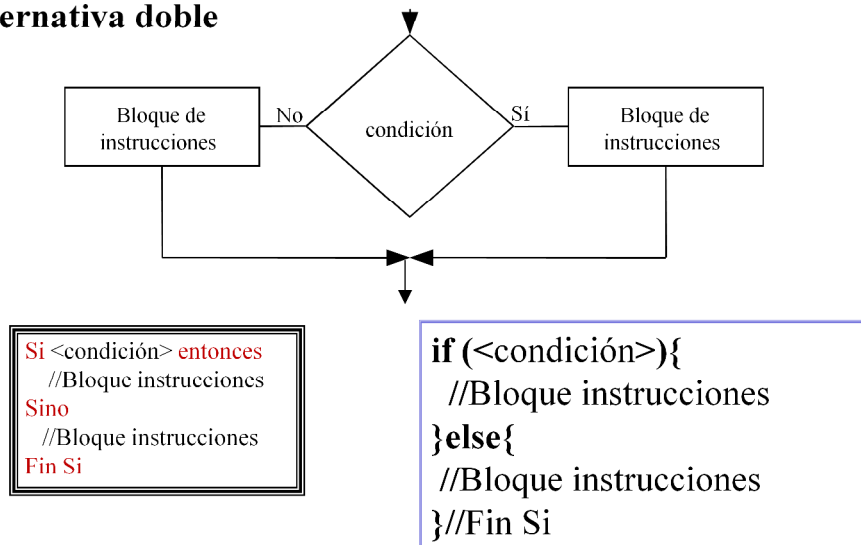
```
Si <condición> entonces  
//Bloque instrucciones  
Fin Si
```

```
if (<expresiónBooleana>){  
    //Bloque instrucciones  
} //Fin Si
```

7. Instrucciones de control

B. Instrucciones alternativas

➤ Alternativa doble



7. Instrucciones de control

B. Instrucciones alternativas

➤ Alternativa múltiple

Estructura **if ... else if ... else** de Java

```
if (expresionBoolean1) {  
    //Bloque de instrucciones  
} else if (expresionBooleana2) {  
    //Bloque de instrucciones  
} else if (expresionBooleana3) {  
    //Bloque de instrucciones  
} else {  
    //Bloque de instrucciones  
} //Fin Si
```

7. Instrucciones de control

B. Instrucciones alternativas

➤ Alternativa múltiple

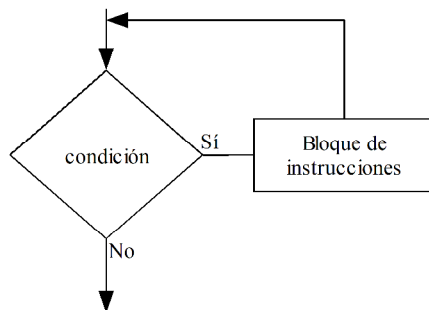
```
Según Sea <nVable> hacer
    <valor>[ o <valor>...] :
        //Bloque de instrucciones
    <valor>[ o <valor>...] :
        //Bloque de instrucciones
    ...
    [Sino
        // Bloque de instrucciones]
Fin Según Sea
```

```
switch (<expresión>){
    case <literal1>:[case <lit2>:]
        //bloque de instrucciones
        break;
    case <literal2>:
        //bloque de instrucciones
        break;
    ...
    [default:
        //bloque de instrucciones]
} //Fin Según Sea
```

7. Instrucciones de control

C. Instrucciones repetitivas

➤ Mientras



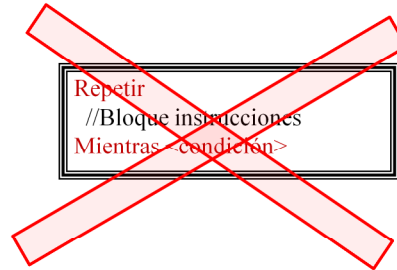
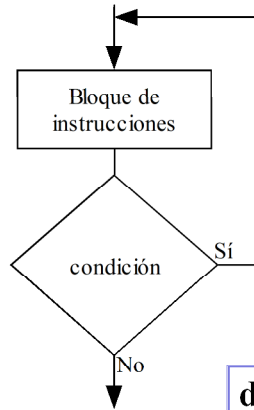
```
Mientras <condición> hacer  
//Bloque instrucciones  
Fin Mientras
```

```
while (<condición>){  
    //Bloque de instrucciones  
} //Fin Mientras
```

7. Instrucciones de control

C. Instrucciones repetitivas

➤ Repetir

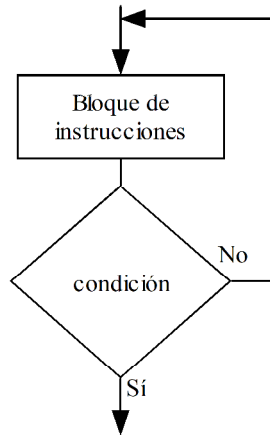


```
do{  
    //Bloque de instrucciones  
}while (<expresiónBooleana>);
```

7. Instrucciones de control

C. Instrucciones repetitivas

➤ Repetir



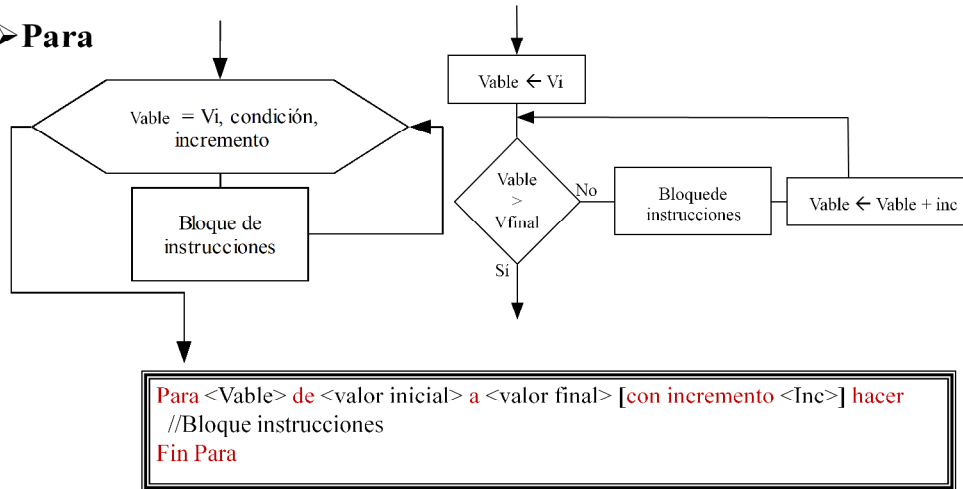
Repetir
//Bloque instrucciones
Hasta <condición>

NO EXISTE EN JAVA

7. Instrucciones de control

C. Instrucciones repetitivas

➤ Para



7. Instrucciones de control

C. Instrucciones repetitivas

✓ Para

```
for (inicialización; expresiónBooleana; incremento){  
    //Bloque de instrucciones  
} //Fin Para
```

Funcionamiento:

